

**MEDIATEK**

# MT8668 Hypervisor Touch User Manual

版本号: 1.0  
出版日期: 2026-01-28

Use of this document and any information contained therein is subject to the terms and conditions set forth in Exhibit 1. This document is subject to change without notice.

---

版本记录

| 版本  | 日期         | 作者  | 描述  |
|-----|------------|-----|-----|
| 1.0 | 2026-01-28 | 杨和成 | 正式版 |

目录

版本记录 ..... 2

目录 ..... 3

1 Touch ..... 4

1.1 概述 ..... 4

1.2 Touch Driver Porting ..... 4

1.2.1 Porting Source Code ..... 4

1.2.2 调试 Touch ..... 5

1.3 Touch Vhost 虚拟化配置---Version1 ..... 6

1.3.1 Touch Vhost 虚拟化跨域性能 ..... 7

1.3.2 Touch Vhost 虚拟化 Touch-Weston 仲裁机制 ..... 7

1.3.3 Touch Vhost 虚拟化 Code 配置 ..... 9

1.3.4 Touch Vhost 虚拟化 Code 配置常见问题 ..... 10

1.4 Touch Handler 虚拟化流程---Version2 ..... 10

1.4.1 YOCTO 端流程（handler） ..... 11

1.4.2 Android 端流程（vhost） ..... 12

1.4.3 新增 TP 操作 ..... 12

1.5 Virtio 虚拟化 Touch 常见问题 ..... 12

1.6 Virtio 虚拟化 Touch 与 Panel 绑定 ..... 13

附件一 附加条款 ..... 14

# 1 Touch

## 1.1 概述

源 Touch driver 的移植一般按照厂商提供的规格进行调试即可，其主要关注的是 I2C 和 GPIO 的配置。对于接 SerDes 的 panel，需关注 SerDes 透传配置。若涉及 Hypervisor 系统，则需要在源 driver 基础上通过虚拟化方式实现 Touch 功能。MT8668 主要有两类移植方案，即 Touch Virito 虚拟化方案与 Touch Vhost 虚拟化方案，其中 Vhost 虚拟化为一般采用方式，Virito 虚拟化方案因性能问题作为备用方案。

## 1.2 Touch Driver Porting

### 1.2.1 Porting Source Code

从联发科技或第三方厂商拿到 Touch Vendor source code 后，一般需如下步骤，确保 Touch driver 本身可以正常工作：

1. 拿到 source code 时，可能存在 kernel 版本升级情况，这时需先移植到对应版本，确保可以编译成功。主要更改文件 Touch driver file、deconfig、DTS、ko\_table，若具体平台需要配置其他文件，请咨询系统工程师帮助。
2. 依据原理图，确认好对应 Touch 所使用的 interrupt/reset pin 的 GPIO，将所使用的 GPIO 号对应配置到 DTS 文件中。
3. 依据 Touch driver 中 compatible 字符，将其对应写到 DTS Touch 节点中。

参考 DTS 配置如下：

```
&i2c0 {
    clock-frequency = <100000>;
    pinctrl-names = "default";
    pinctrl-0 = <&i2c0_pins>;
    status = "okay";

    ilitek@41 {
        compatible = "ilitek,touch";
        reg = <0x41>;
        interrupt-parent = <&pio>;
        interrupts = <9 0x0>;
        ilitek,irq-gpio = <&pio 9 0x0>;
        ilitek,reset-gpio = <&pio 60 0x0>;
        ilitek,vbus = "vcc_i2c";
        ilitek,name = "ilitek_i2c";
    };
};
```

其中,

- i2c 0: Touch 挂载的节点位置
- clock-frequency = <100000>; i2c clk 大小, 具体大小请参照 spec, 公版默认设置为 400k 高速模式
- compatible = "ilitek,touch"; Touch 匹配字符
- interrupts = <9 0x0>; Touch 中断 GPIO
- ilitek,reset-gpio = <&pio 60 0x0>; Touch reset GPIO

## 1.2.2 调试 Touch

常用调试命令:

### 1. 显示 Touch 轨迹线:

```
settings put system pointer_location 1
settings put system show_touches 1
```

### 2. 查看 log:

```
echo 1 > /proc/mtprintk
```

### 3. 查看 GPIO 引脚状态:

```
find -name soc.pinctrl
cd /proc/mtk_gpio
cat soc.pinctrl
```

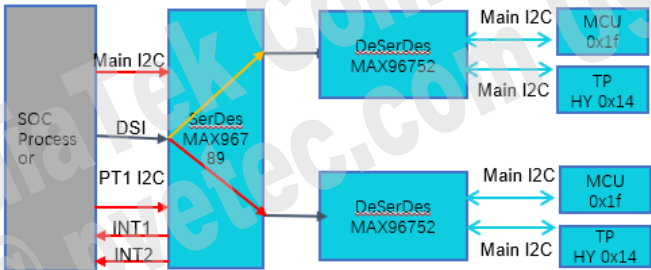
### 4. 查看 ko 是否正常加载:

```
Lsmod |grep touch_name
```

在将 Touch source code 移植到平台并可以编译成功后, 如平台启动后, Touch 未生效, 一般需要注意如下点:

1. Touch 驱动 probe 流程 是否有跑完, 一般驱动无法跑完有如下几种类型错误
  - DTS 未匹配, 此问题需检查 DTS 中是否有 enable 此 Touch 节点, compatible 字符是否匹配
  - I2C 返回 -6, 此错误表明未能识别 Touch 设备, 需检查硬件线路是否连接正常, 是否供电正常, 如检查无误后需要量测 I2C 波形进一步理清
  - I2C 返回 -110, 需检查 I2C 部分是否已正常工作
2. 概率性 Touch 不生效, 一般 Touch 的供电电压有具体的限制, 可以在 Touch driver probe 时, 打印电压值来理清问题
3. Touch 位置存在偏差, 需根据 display size 进行适配 Touch
4. Touch 模块存在漏电问题, 判断 Touch 在 suspend 时候是否有主动断电, 及拉低对应 GPIO
5. 检查 interrupt 和 reset pin 上电时序, 特定的 Touch driver 对两者的时序有具体要求, 需按规格配置

6. 配置 SerDes, 查询 SerDes 手册, 联发科技的公版现已可按如下方式支持 6 panel Touch

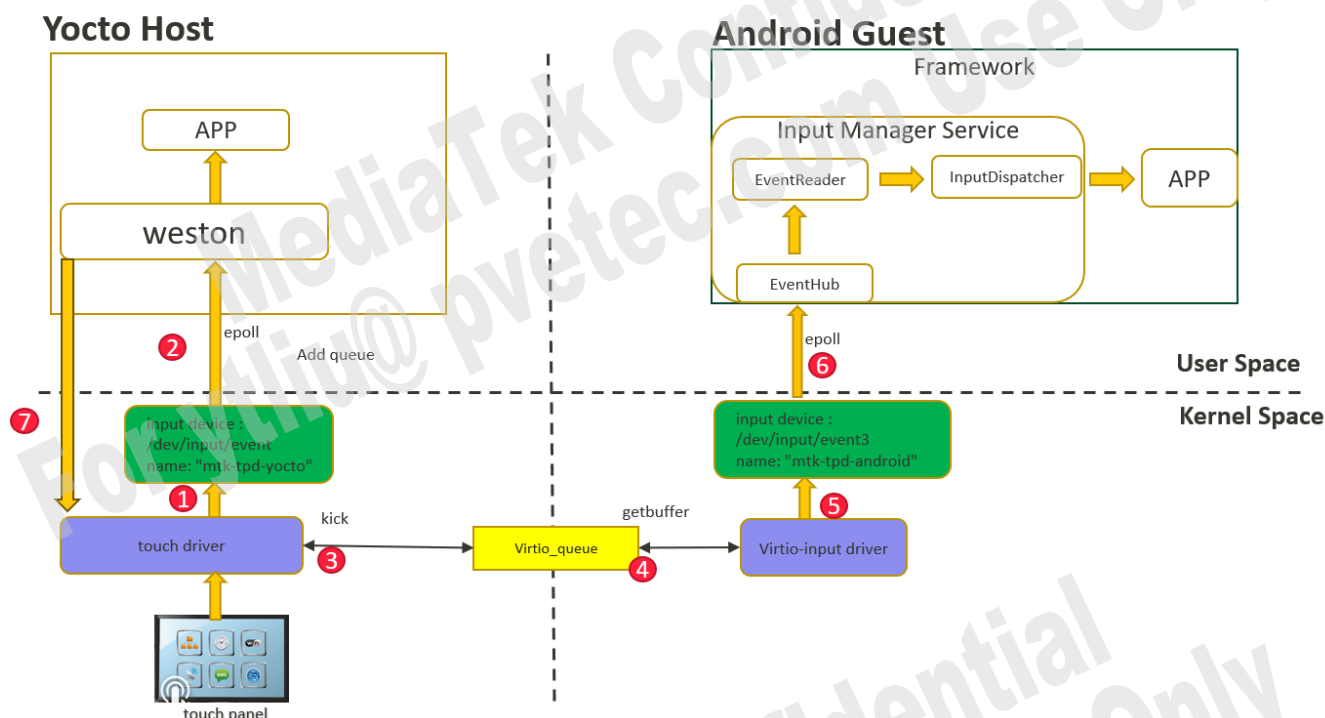


参考 SerDes 配置如下, 需参考 96789 及 96752 手册对应配置:

```
superframe_setting_hy: setting3 {
    ser-super-frame = <1>;
    ser-init-cmd = <
        0x02dc 0x04 0x00 /* 96789 INT: des gpio4-> ser gpio10*/
        0x02dd 0xaa 0x00 /* 96789 huayang tp ser int GPIO10 */
        0x02de 0x6a 0x00 /* 96789 huayang tp ser int GPIO10 */
        0x02d9 0x04 0x00 /* 96789 INT: des gpio4 -> ser gpio9 */
        0x02da 0xa9 0x00 /* 96789 huayang tp ser int GPIO9 */
        0x02db 0x69 0x00 /* 96789 huayang tp ser int GPIO9 */
    >;
}
```

1.3 Touch Vhost 虚拟化配置---Version1

为 Touch 安全性, 有将 Touch 放置在 Yocto 端, 这种情况就需要 Touch 虚拟化来完成, 公版 Touch Vhost 虚拟化流程概图如下所示:



1. 触摸 panel，产生中断信息，通过 Touch driver 上报 Touch firmware 解析的触摸位置信息。
2. Yocto 端 weston 接收位置信息，由 weston 来决定是否将触摸事件分发给 Yocto APP。
3. Touch driver 通过 Vhost 建立的 virtio\_queue 将 event data 发送到 Android 端。
4. Android 端虚拟 Touch driver 接收 Yocto 传过来的 event data。
5. 虚拟 Touch driver 将 event data 组合后，上报给 input 模块。
6. Input 模块对 event 信息处理后发往 framework 层进一步处理。
7. 通过 Weston 传信息，Touch driver 将决定 event 发给 Yocto/Android 哪一端。

### 1.3.1 Touch Vhost 虚拟化跨域性能

一般情况下，我们会认为在 Touch 虚拟化实现中，跨域耗时会占较大的时间比重，在 Virtio 虚拟化时确实是这样，但当我们采用 Vhost 虚拟化后，从 Yocto 发往 Android 端的耗时可以降低到 1ms 以下，在轻负载的情况下，平均耗时在 100μs 附近，性能远优于 Virtio 虚拟化方式。

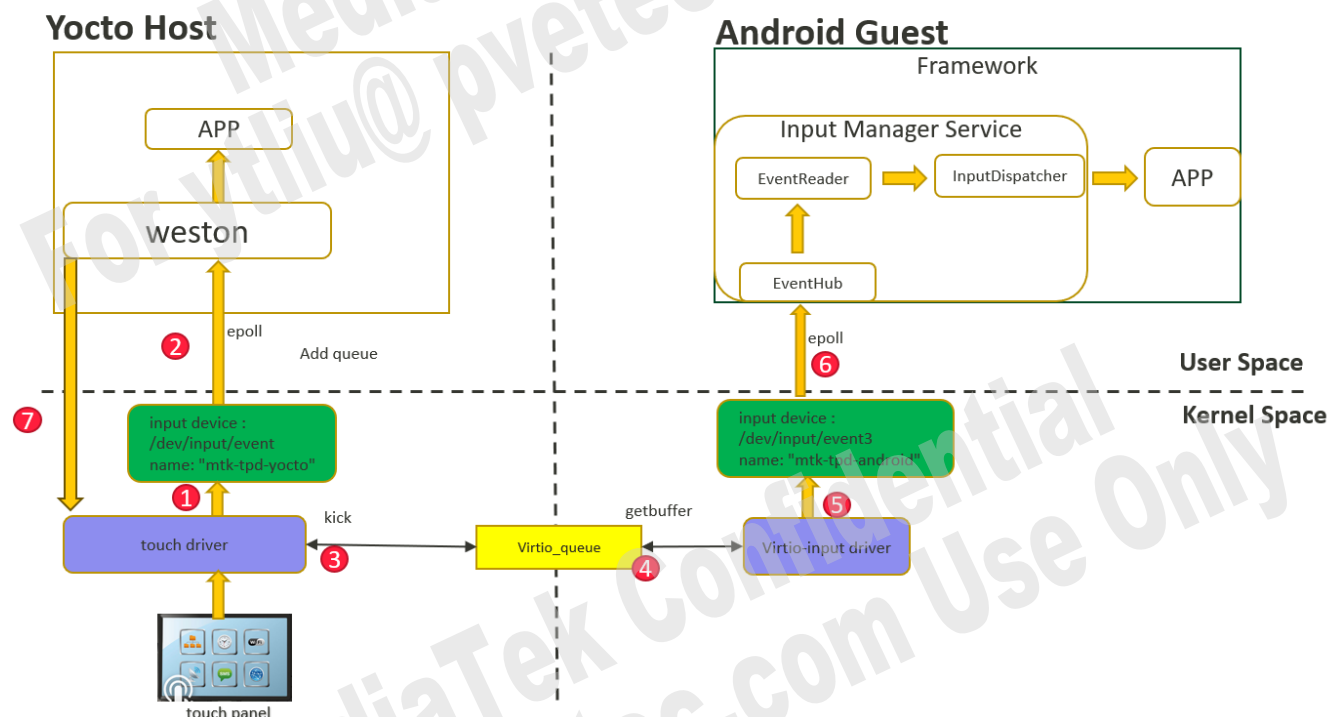
### 1.3.2 Touch Vhost 虚拟化 Touch-Weston 仲裁机制

需求背景：在 Android display 上出现 Yocto 的 APP，Touch 需要有选择性的发往一边。

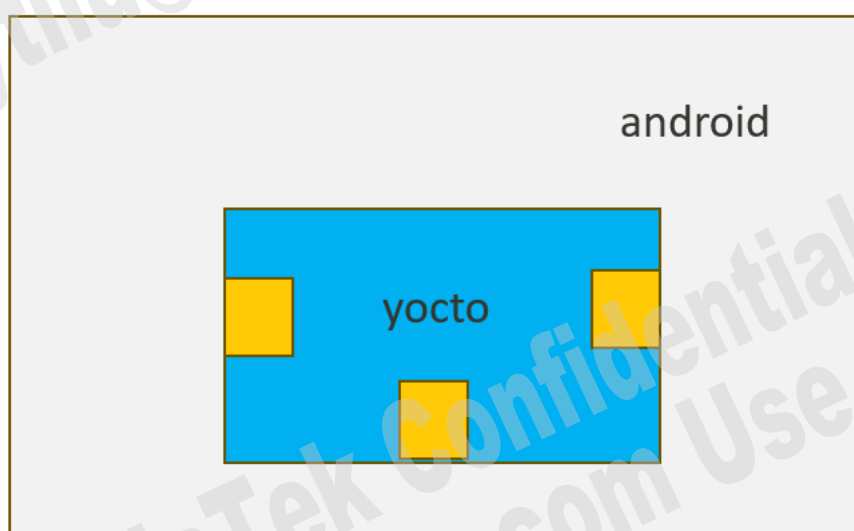
方案选择：对于 Yocto APP 位置是否固定，提供两种方案。

方案一：对于 Yocto 的 APP 固定位置时

当 Yocto APP 启动后，Weston 会感知到 APP 的位置信息，然后提前将位置信息下发给 driver，当触摸事件产生后，Touch driver 将依据触摸的 (x,y) 位置信息和从 Weston 发来的位置区域信息，最终决定将 Touch event 发往哪一端：

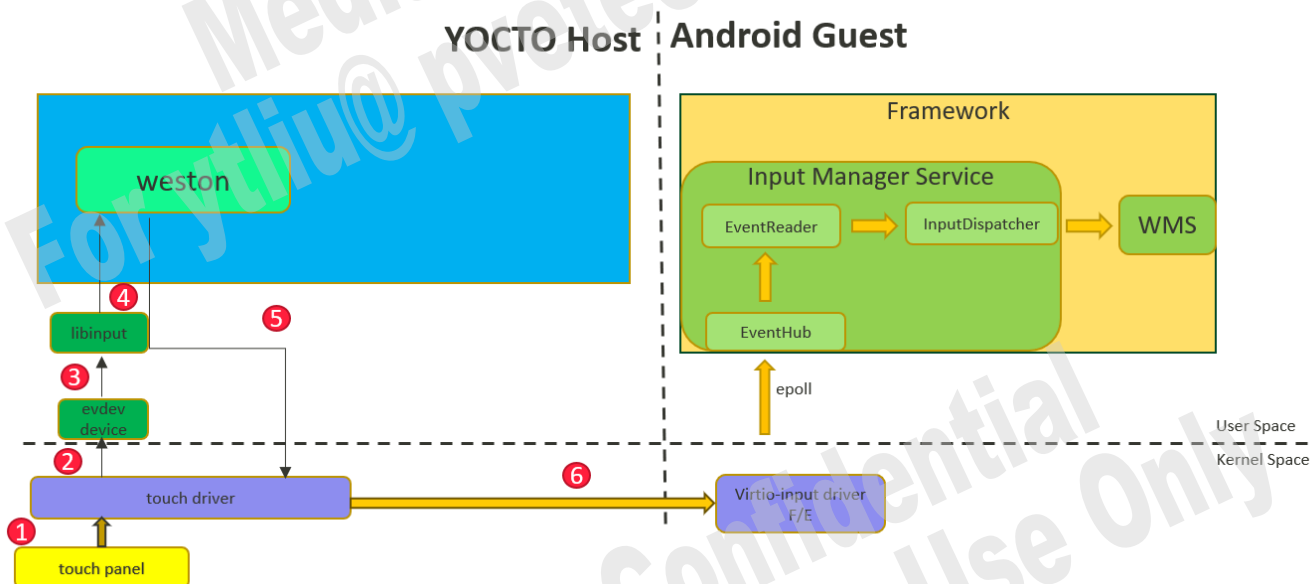


但此方案是将业务逻辑下发到 Touch driver，无法处理 Weston input\_range 这种情况，也无法满足从 Yocto 进行数据注入的情况。



无法处理 weston input\_range 这种情况，黄色框为 Yocto 区域，但是这块区域会留给 Android 响应。

方案二：每次点击，Touch driver 都会上报第一个 event 中的 down 和 (X+Y) 给到 Yocto，然后做等待动作，直到 Weston 下发发往哪一端的 flag，如果在 time out 时间内没有等待到这个 flag，Touch driver 会默认将 event 发往 Android。



以第一个 down 作为判断发往 Android/Yocto OS 的判断点，直到 up 前的所有 event 都将会发给原 OS。业务逻辑放在 Weston。

### 1.3.3 Touch Vhost 虚拟化 Code 配置

Touch Vhost 虚拟化与源 Yocto Touch driver 强相关，对于不同的 Touch driver 配置上有细节差异，需针对 Touch driver 进行二次配置。Touch Vhost 虚拟化本质上是提供一个 virtio\_queue 通道，这个通道提供将 data 从 Yocto 发往 Android 的能力，我们将 Touch driver 的每一次 event 信息都保存成一组 data，将这组 data 传给 Android 后，由 Android 端虚拟化 Touch driver 拿到这组 data 进行重新组装 event，然后将 event 信息发往 framework 层。

#### 1. Yocto 端 Touch driver 配置细节

对于 Yocto 端的 Touch driver 原有流程保持基本不变，在原有流程的 touch down/touch release 后边添加类似如下 code，同步将 Touch event 发往 Android。

```
INIT_WORK(&work->work, touch_event_work_func);
sprintf(work->str, "%d %d %d %d %d %d %d", flag, x, y, h, w,
        id, touch_type);
queue_work(touch_event_wq, &work->work);
```

需要保证 Touch driver 中发送 data 的时序性。

## 2. Android 端 Touch driver 配置细节

参照公版已有建立的虚拟 Touch driver，建立与 Yocto 相对应的虚拟 Touch driver，Yocto 和 Android 两边的联系通过 Touch\_type 建立。

## 3. Hypervisor 端 Touch driver 配置细节

使用公版已有 code 即可，无需另外配置。

### 1.3.4 Touch Vhost 虚拟化 Code 配置常见问题

需要注意第一次配置时，需在 Touch driver 中尽量加全 log，避免多次添加耗时，调试报点逻辑时经常性会遇见各种细节逻辑问题。

#### 1. 配置完后，Touch 多指标表现异常

注意源 Touch driver 使用协议为 Type A 还是 Type B，不同协议 Yocto 与 Android 端的配置逻辑有差别，例如是否添加 mt\_slot 参数。

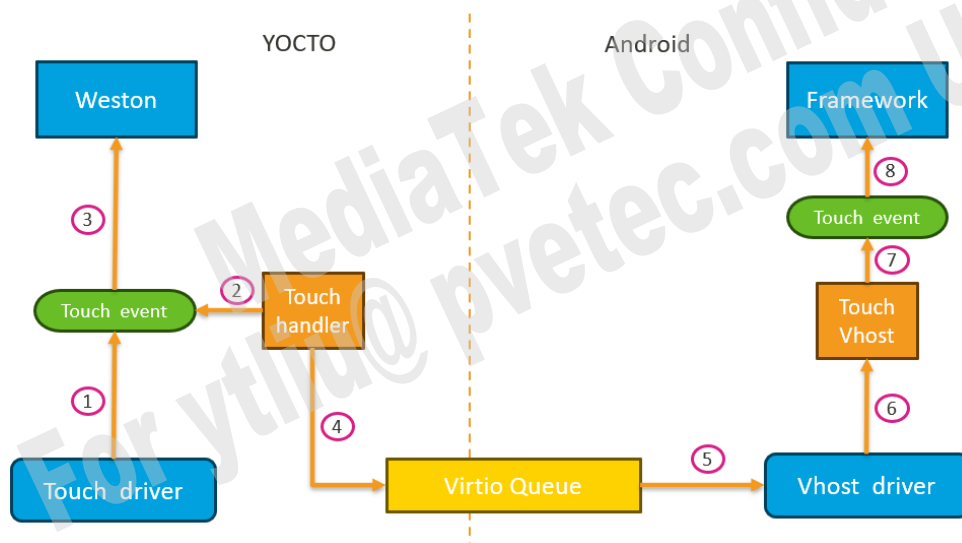
#### 2. 配置完成后，Down 点击不生效，但滑动无问题

着重检查是否为时序问题，公版中通过 wait\_for\_completion\_timeout(&touch\_event\_done, 3); 这个 API 来保证。

### 1.4 Touch Handler 虚拟化流程---Version2

MT8668 主要采用的是 Version2 方案。Version2 方案是根据 1.3 中提到的 version1 方案的基础上来进一步优化处理的。

当点击屏幕时，Touch driver 会上报 event 事件给 Weston，这个过程中 Touch handler 会去监听 event 事件，如果是需要上报给 Android 的，那么就会通过 virtio\_queue 通道将 data 发送给 Vhost driver，然后就会调用 Touch Vhost 对 data 进行组装，最后上报 event 给 framework 层。



Touch Vhost 是根据 Touch handler 发送 data 的 touch\_type 进行选择不同设备处理接口的，在接口中又根据 data 中的 flag 来进行组装 event 事件进行上报。

#### 1.4.1 YOCTO 端流程 (handler)



##### 1. 注册 handler

- 1) 调用 `input_register_handler(&my_input_handler)`。
- 2) 内核会把你的 handler 注册到输入子系统。

##### 2. 匹配设备

- 1) 内核会用 `my_ids (id_table)` 去匹配系统中的 input 设备。
- 2) 匹配成功后，调用 `my_input_connect` 对设备进行连接。
- 3) 并且将设备名与监听设备名进行对应绑定

##### 3. 事件处理

- 1) 设备有事件发生时，内核会调用 `my_input_event` 处理事件。
- 2) 首先会判断监听 event 事件类型然后对其进行对应处理，并且设置 event 对应的 flag 方便安卓端进行解析。

##### 4. 设备断开

- 1) 设备断开，内核会调用 `my_input_disconnect` 做清理，进行 handler 注销。

##### 5. 注销 handler

- 1) 调用 `input_unregister_handler(&my_input_handler)`; 注销 handler。

### 1.4.2 Android 端流程 (vhost)

1. 在 `touch_virtul` 中会进行接收 Yocto 发送的数据并且根据 `touch_type` 对不同的设备进行不同的处理
2. 以 mipi 小屏为例, 根据不同的 flag 进行不同的 event 事件上报. 参考 `report_touch_down_mipi`

### 1.4.3 新增 TP 操作

#### 1. YOCTO 端(input\_handler)

- 1.1. 在 `touch_info` 中添加你的监听设备名 (自定义)
- 1.2. 同时需要在 `my_ids` 数组中做对应的添加
- 1.3. 在 `my_input_connect` 中进行新增设备屏幕与其监听设备名的绑定
- 1.4. 在 `my_input_event` 中进行设备处理的添加 (一般只需要根据监听设备名添加 `touch_type` 设备类型)

如果你需要添加其他的上报 event 事件需要进行定义 flag, 与其他 event 做好区分即可。

注: 设置的 flag 不能为 0, 否则 android 端无法正常接收

#### 2. Android 端(tpd\_control)

- 2.1. 首先定义 TP 输入设备和端口号 (如: `static struct input_dev *touch_input_dev_mipi,`  
`#define touch_input_dev_mipi_port 0.` 注: 端口号与 `touch_type` 是一致的)
- 2.2 在 `touch_input_init` 中进行虚拟设备节点的创建
- 2.3 同时在 `touch_virtul` 中进行对应根据 `touch_type` 设备类型进行不同操作
  - MIPI 小屏参考 `report_touch_down_mipi` 进行处理
  - 华阳屏参考 `report_touch_down_hy_dsi_0` 进行处理

## 1.5 Virtio 虚拟化 Touch 常见问题

#### 1. Touch 虚拟化端未生效

在 Yocto 和 Android 端分别输入 `getevent -1`, 触摸屏幕, 如:

- Yocto 端无报点数据产生, 需检查源 Touch driver 配置
- Android 端无报点数据产生, 检查 Touch 虚拟化是否有 `enable`、是否有设置了对应的 Touch 节点名字

#### 2. 报点位置与 display 存在偏差

在 Android 端输入如下命令，会显示出 Touch 的轨迹线，看是否有偏差及偏差规律。

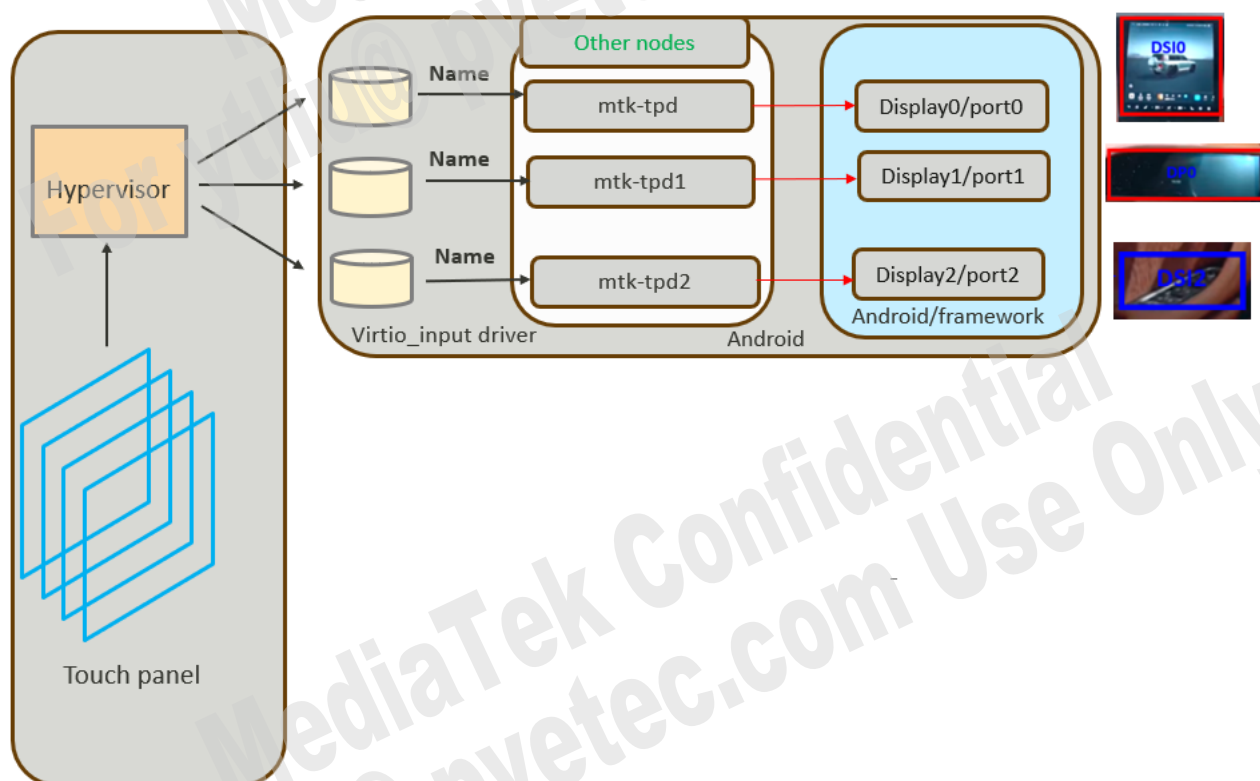
```
settings put system pointer_location 1
settings put system show_touches 1
```

在 Android 端输入 `getevent -i`，查看 Touch raw\_size 是否和 display size 对应，如不对应，需调整 NBL 中的 display\_size 信息。

3. 有多个需要虚拟化的 Touch 时，Touch raw\_size 应用到错误的数据  
需要注意在配置 `uos_alps_pv8668.lua` 时，display\_size 的配置顺序与 Touch driver 加载的顺序一致。
4. 在重载多指快速滑动场景下，可能会出现 Android 端报点丢失的情况，这种情况公版有遇见，且排查为 Android 端 `virtio_input` 原生 driver 处报点会被 input 模块过滤，且 `virtio_input` 与 input driver 均为谷歌原生 driver，暂无法修改，建议采用 Vhost 虚拟化规避此问题。

## 1.6 Virtio 虚拟化 Touch 与 Panel 绑定

如下图，input 模块通过节点的 location 信息将 Touch 节点与 panel 进行绑定，当有其他节点生成在 Touch 节点前时，会出现 location num 变化导致绑定失效的问题，需通过 `ko_table` 保证如下事项，如后续有新加载驱动，并此驱动有生成节点，则此新驱动需放置在 Touch 节点后，避免绑定异常。



## 附件一 附加条款

您(或您所代表的公司或其他法律主体, 以下合称「您」)是否得取得、下载与使用本文件及其相关信息皆应以您接受本附加条款为先决要件。您对于本文件之使用、取得或下载即表示您已接受本附加条款并同意受本附加条款之拘束。若您不同意受本附加条款之拘束, 您将不得使用、取得或下载本文件并应立即删除或毁弃所有本文件之副本。

本文件含有联发科技股份有限公司及其关联公司(以下合称「联发科技」)或其授权人之机密信息及专有信息, 仅供您为本文件所描述之联发科技芯片组作内部使用而不得用于其他任何目的(包括但不限于确认或提供支持任何对联发科技、其供货商及/或其直接或间接客户所提潜在专利侵权主张的证据)。禁止未经授权使用或揭露本文件及其中所含信息。您同意赔偿联发科技因您未经授权使用或揭露本文件及其中所含信息之一部或全部导致联发科技所受之任何损失或损害。

联发科技及其授权人保有本文件的所有权及组成所有权之各项权利且未针对任何知识产权授权(无论明示或默示、透过禁反言原则或其他方式)。联发科技得随时变更本文件内容而无须另行通知。联发科技无须承担与使用或信赖本文件相关或因使用或信赖本文件致生之任何责任, 包括但不限于间接损害或附带损害赔偿責任。

本文件及联发科技所提供关于本文件之任何其他材料、信息或技术支持, 均以现况交付为准, 联发科技不负任何明示、默示、法定或其他形式之担保责任, 特别是适销性、不侵权、针对特定用途之适用性、完整性或正确性之担保责任或任何由贸易惯例或交易、履行过程所生之担保责任。对于联发科技为符合您所提规格或遵循特定标准或标准组织的要求所作之交付物, 联发科技亦不承担任何责任。

于未对前述条款造成限制之情形下, 联发科技不对其产品任何特定用途之适用性承担任何保证或担保责任, 亦不承担任何因产品、电路或软件之应用或使用致生之赔偿责任。您同意您应单独承担关于整合您的产品与联发科技产品所涉及之设计、验证与测试之所有责任, 并确保前述整合产品符合相关标准及任何安全性要求或其他要件。

本附加条款及所有与本附加条款或本文件相关之行为应以台湾法律管辖、解释与阐明, 不适用冲突法原则。